# GPU kernels on real matrices

What limits dense GEMM, sparse SpMV and conjugate gradient on a GPU — and when the GPU actually pays off



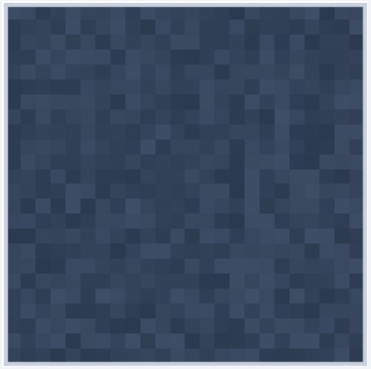

GeForce RTX 5090 · CUDA 12.9 · CuPy 13.6.0 | Numerical Libraries Mini-Hackathon

PROBLEM

# What we multiply, and what could limit it

Five inputs, three kernels — is each one limited by arithmetic, by memory bandwidth, or by latency?

A · GEMM

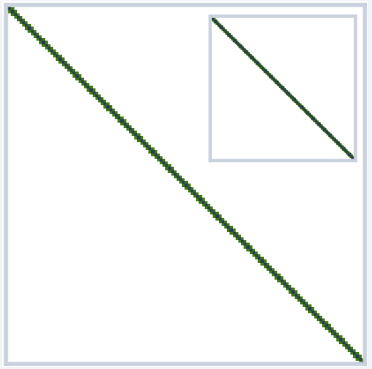


### Dense n × n

n = 128 … 16,384

every entry stored

B · SpMV

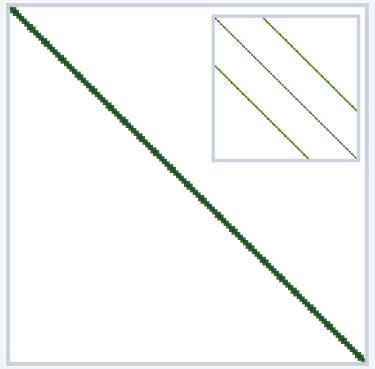


### cant*

62k rows · 65 nnz/row

banded, FEM-like · SPD

B · SpMV

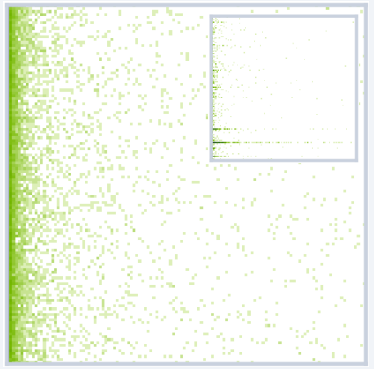


### ecology2*

1M rows · 5.0 nnz/row

2-D grid stencil · SPD

B · SpMV

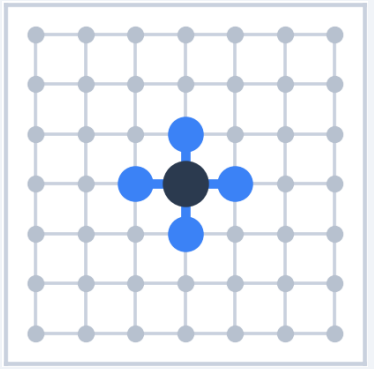


### webbase-1M*

1M rows · 1.7 nnz/row

web graph · power-law rows

C · CG



### 2-D Poisson

n = 256 … 4.2M

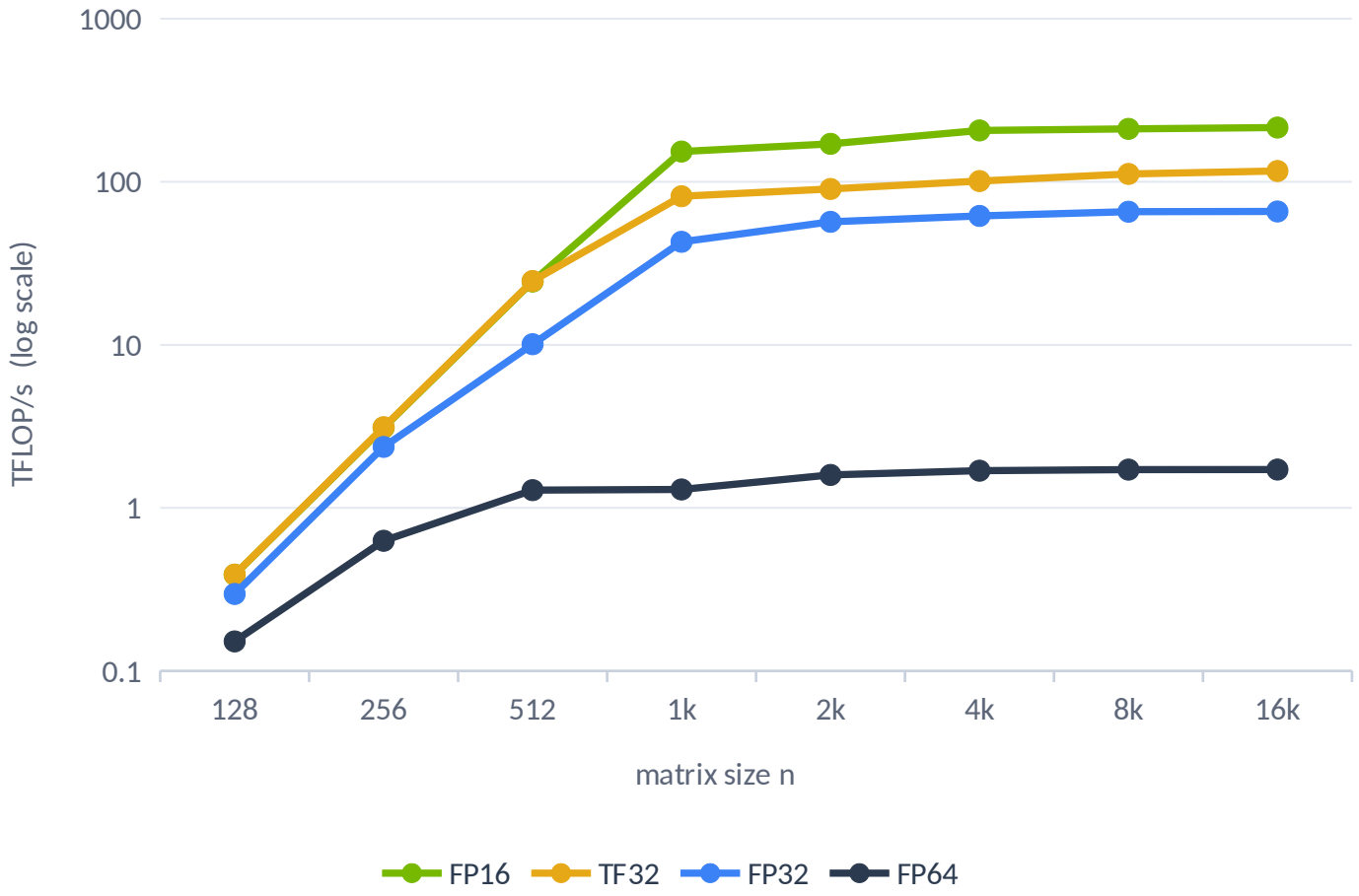
5-point stencil · SPD

** synthetic stand-ins with the size and sparsity class of the SuiteSparse matrices (download failed on the run node) · insets: first 3,000 rows*

**FLOP per byte decides the limit:** GEMM **grows with n** · SpMV **≤ 1/6** · CG ≈ 0.11 + latency · ridge of this GPU **≈ 1.1**

# GEMM is compute-bound: precision is the lever

cublasGemmEx, n × n × n, data resident on the GPU — 215 TFLOP/s in FP16 vs 1.7 TFLOP/s in FP64



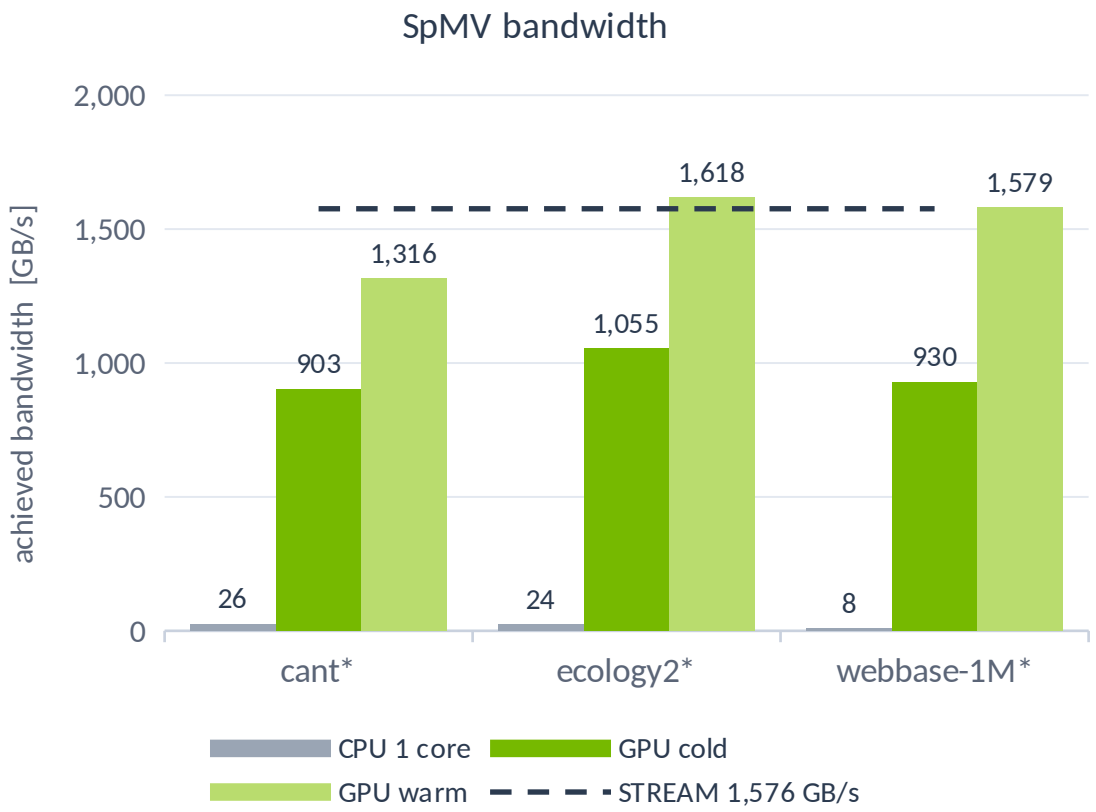

**Precision ladder at n = 16,384**

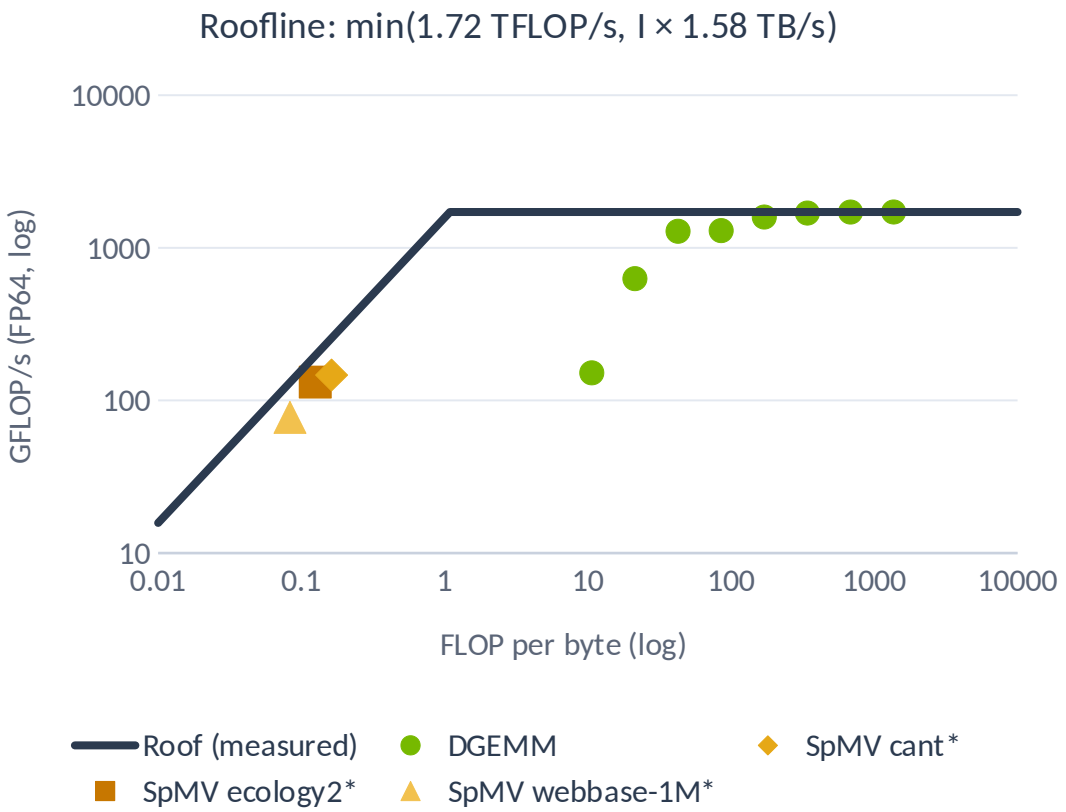
| Precision | TFLOP/s | vs FP64 | rel. error |
|---|---|---|---|
| FP64 | **1.72** | 1× | reference |
| FP32 | **66** | 38× | $2.7\times10^{-7}$ |
| TF32 | **116** | 68× | $2.9\times10^{-4}$ |
| FP16 | **215** | 125× | $3.6\times10^{-4}$ |

- Small n is launch-bound: TF32/FP16 calls with n ≤ 512 all take ≈ 11 µs
- Tensor cores: TF32 1.8× and FP16 3.3× over FP32, for ≈ 3 fewer correct digits
- FP64 runs at the consumer 1:64 rate, so SGEMM is 38× faster than DGEMM

# SpMV is bandwidth-bound, GEMM compute-bound

cuSPARSE SpMV reaches 0.90–1.06 TB/s (57–67% of measured bandwidth) and is 35–116× faster than one CPU core

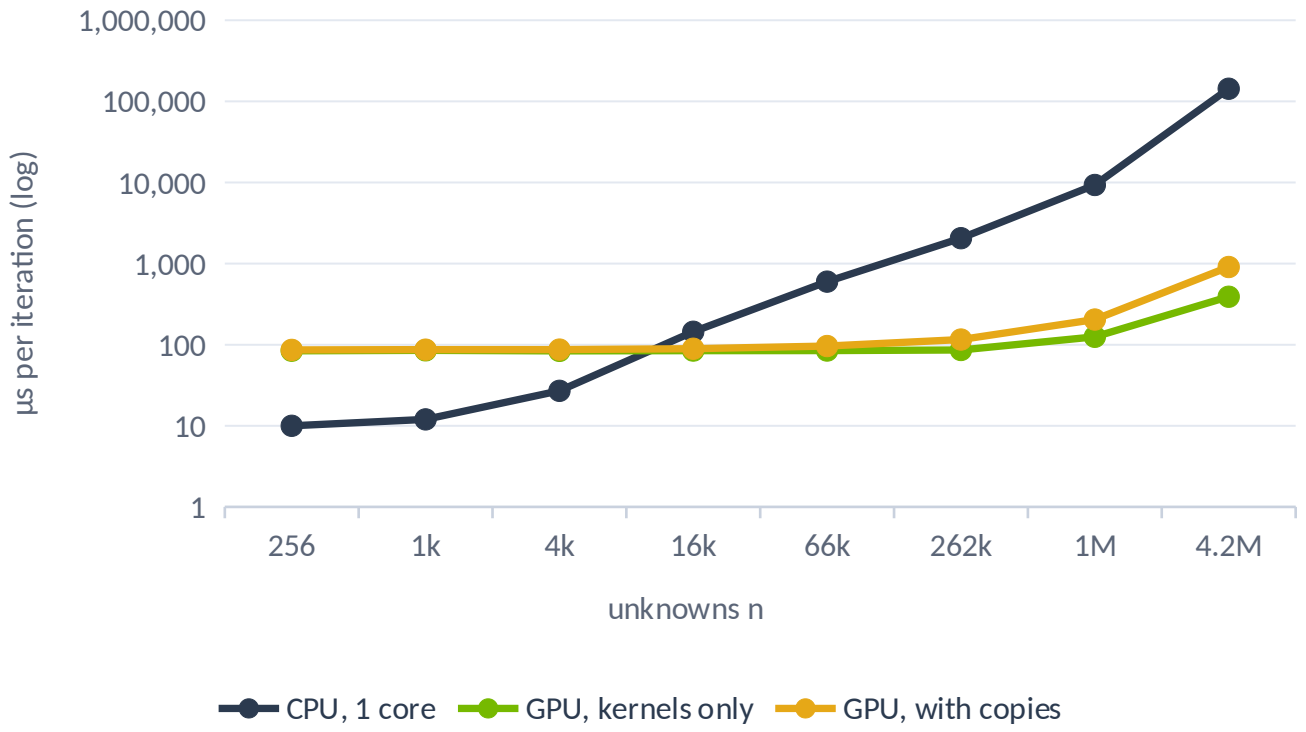



**Reading:** SpMV sits at ≈ 0.1 FLOP/B, left of the ridge (1.1): most of the bandwidth, only 5–9% of DGEMM's FLOP/s — fewer bytes help, faster arithmetic does not.

# CG: the GPU wins above n ≈ $10^4$

2-D Poisson, 100 iterations · cuSPARSE SpMV + cuBLAS dot/axpy vs SciPy/NumPy on one core




## $n^* \approx 1.05 \times 10^4$

GPU iterations cost a flat ≈ 85 µs (launch + sync latency) until n ≈ 250k; one core streams ≈ 24 GB/s. Equating the two predicts $1.1 \times 10^4$.

**Copies: speed-up, kernels only → with copies**

**cant*** 23 its

**23× → 9.0×**

copies = 60% of GPU time

**ecology2*** 500 its

**96× → 85×**

copies = 12% of GPU time

**TAKEAWAY** **FLOP per byte decides what to optimise:** GEMM → precision · SpMV → bytes · CG → latency, keep data on GPU